

Audion Disk Tools - User Guide

Current for the project state on 2026-07-06.

Audion Disk Tools is a portable-first Windows toolkit for file routes: audit, compare, one-way copy, BACKUP-MIRROR, two-way sync, archiving, network transfer, SMB share helpers, and native RClone operations. The GUI is a Workbench over the same CLI: choose **Source**, **Target**, an operation, and parameters; the command runs through the portable runtime with live terminal output on the right.

Documentation Map

- [README_EN.md](#) - compact feature map.
- [USER_GUIDE_RU.md](#) - Russian guide.
- The detailed GUI reference is consolidated into this guide below.
- [COMMAND_REFERENCE_RU.md](#) - complete command and parameter reference in Russian.
- [RCLONE_OPERATIONS_RU.md](#) - RClone cloud layer.
- [RCLONE_PORTABILITY_RU.md](#) - RClone install, portable/system config, import/export.
- [ARCHIVE_OPERATIONS_RU.md](#) - archives, SFX, encryption, verification, and optional source deletion.
- [MANIFEST_REFERENCE_RU.md](#) - Manifest / Diff, Source -> Target path lists, and service checksums.
- [PDF_EXPORT_RU.md](#) - Markdown-to-PDF export into **docs\PDF**.
- [SMOKE_TEST_CHECKLIST_RU.md](#) - verification checklist.

Start

```
launcher_gui.cmd
```

The GUI with the right-side terminal starts only on **127.0.0.1**, **localhost**, or **:::1**. Non-loopback hosts are refused unless **AUDION_ALLOW_REMOTE_GUI=1** is set explicitly. The desktop wrapper does

not request UAC by default; use `--elevate` or `AUDION_ELEVATE=1` only when an operation really needs administrator rights.

CLI launchers:

```
launcher_project.cmd  
launcher_profiles.cmd
```

Direct CLI:

```
runtime\python.exe system_core\main.py info  
runtime\python.exe system_core\main.py pairs
```

Build or repair the portable environment:

```
builder_main.cmd
```

Core Model

Source is the truth side for one-way and mirror flows. **Target** is the receiving or backup side. Both can be any path visible to Windows/Python: project **input/output**, external drives, UNC paths, mounted folders, cloud-client folders, or ordinary local directories.

The GUI does not replace the CLI. It prepares parameters, calls the same services/CLI, streams stdout/stderr into the right terminal, and writes logs/reports to disk.

Main Operations

BACKUP-MIRROR copies/updates from source to target, verifies earlier phases, then removes target-only files. This is a hard mirror workflow.

One-way copies new and changed files from source to target. Target-only files remain.

Two-Way sync exchanges missing or newer files in both directions. It does not automatically delete files.

Masks is a manual glob-mask constructor and can run BACKUP MIRROR, ONE-WAY, or TWO-WAY. In the mask constructor, BACKUP MIRROR uses **mirror_safe**, which moves target-only files into **_audion_quarantine** instead of hard-deleting them.

`Saved profiles` are loaded from `config\sync_pairs.json`. If that file has no pairs, the GUI displays bundled examples from `config\sync_pairs.example.json`.

`Archiving` packs the selected source into the selected target. The main format row is ZIP, 7Z, SFX, TAR, TAR.GZ, and TAR.ZSTD when the backend is available; TAR.LZ4 appears only when `lz4.exe` is found.

`DATA TRANSFER` is one section for folders, shares, servers and clouds alike: five operations, a list of destination machines, and an engine chosen per pair — Robocopy or RClone.

`Manifest / Diff` works over the current Workbench `Source -> Target` route: choose extension groups when needed, build a Diff, then apply it as a one-way run or a mirror. Separate `__CHECKSUMS__.b3` files for source/target remain service integrity checks.

Safety Terms

`mode=safe` is the normal default for real workflows.

`mode=quick` is faster and more metadata-driven.

`operation_policy=copy_update` copies/updates and keeps target-only files.

`operation_policy=mirror_safe` moves target-only files into `_audion_quarantine`.

`operation_policy=mirror_hard` deletes target-only files after successful earlier phases.

`anchor=true` in a profile requires the `.audion-anchor` file at both roots before scanning. It matters wherever a root comes from the network: a disconnected mount presents as an existing but empty directory, and without the anchor such a run reads as "delete everything in target". The anchor is written once by an explicit command and is never created during a run:

```
runtime\python.exe system_core\main.py anchor N:\Projects
```

Above the command fields the GUI shows a `PREFLIGHT` card: anchor state, free space against planned bytes, and planned removals as a count and as a share of the target. For mirror policies it also shows a confirmation token that binds the apply to the preview the operator saw. See [SYNC_PROFILES_RU.md](#).

RClone Workflow

RClone uses the discovered `rclone.exe`, usually `Tools\rclone\rclone.exe`. The compact `Installer RClone` row has `System` and `Portable`: `System` runs `install\Install-System-Rclone.cmd`, installs user-scope RClone into `%LOCALAPPDATA%\Programs\rclone`, and adds it to the user PATH; `Portable` runs `install\Install-Portable-Rclone.cmd` and installs/updates `Tools\rclone`. Both installers download the latest stable Windows AMD64 ZIP, verify `rclone.exe version`, and do not overwrite `rclone.conf`.

By default the GUI uses `Portable` config scope: commands get `--config <ROOT>\config\rclone\rclone.conf` and `--cache-dir <ROOT>\tmp\rclone-cache`. The `Portable/System/Custom path` selector lives in `Configuration`, because it chooses `rclone.conf`, not the install target. `System` scope uses the normal `%APPDATA%\rclone\rclone.conf`; `Custom path` accepts an explicit `rclone.conf` path. `Configuration / Import` copies system config to portable with backup, `Export` copies portable config to system with backup, and `Doctor` performs a read-only active config/remotes/absolute-path check. The full mechanics are documented in [RCLONE_PORTABILITY_RU.md](#).

First-use flow:

1. Install RClone if needed.
2. Run `Version`.
3. For Yandex/Google Drive/OneDrive/Dropbox/Box/Mega, use `OAuth providers`; the GUI opens a clear RClone helper window for provider authorization. The default provider is Google Drive, with remote name `drive`.
4. For servers, use `Create/update SFTP remote`; fill `user@host:port`, remote path, and auth method.
5. Use full `Config` only for rare backend cases.
6. Run `List remotes`.
7. Run `About remote` or `Test remote`.
8. Run `Copy endpoint route`, `Upload copy`, or `Download copy`.
9. Use `Check endpoint route` or `Check one-way` for verification.

RClone commands are built as subprocess argument lists. The preview uses Windows quoting for display only. OAuth tokens and `rclone.conf` content are not stored in project JSON/YAML; portable `rclone.conf` is a separate ignored file under `config\rclone`.

RClone **Config** and **GUI** open directly in a separate visible console. Use **DEBUG** logging only temporarily for local diagnostics: raw rclone logs can contain sensitive remote/request details.

Before running OAuth/SFTP modes, hover **RUN**: the tooltip explains which window opens, what to leave blank/default, and which **remote:path** is produced after successful authorization.

Universal RClone routes use paired **SOURCE ENDPOINT** and **TARGET ENDPOINT** panels. The primary route is remote -> remote, for example **drive:Project -> server:/home/user/backups**; a Workbench path can be selected explicitly as an endpoint type when a local side is needed. SFTP remotes are built in the GUI as **user@203.0.113.10:22:/home/user/backups**; password auth does not save the password in cache/history, while key/agent modes avoid project-stored passwords entirely.

RClone modes are grouped into **Transfer / Route / Settings** tabs: only the active branch commands remain visible, and the active command is highlighted. SFTP **user** and **host/IP** have a dedicated history dropdown from successful operations; the cache lives in **config\rclone_endpoint_history.json** and contains no passwords.

Progress

The right terminal streams live output. RClone progress is parsed from **Transferred**, **Checks**, **Retries**, **Errors**, speed, and **ETA** lines. The GUI updates status text, percentage, progress bar, and retry/error counters. Transfer profiles include **--stats 1s --progress**, which is important for slow or unstable links. **Bandwidth limit** offers Unlimited, Speed preset, or Custom speed with spinners.

Terminal history does not save or pin commands with obvious secret markers (**token**, **secret**, **password**, **Authorization**, **Bearer**, **-p...**). Archive passwords are entered through masked fields, and archive **-p...** arguments are displayed in the GUI log as **-p[REDACTED]**.

Minimal Verification

```
runtime\python.exe -m py_compile system_core\main.py system_core\auditor_core.py
system_core\services\disk_auditor_service.py system_core\ui_nicegui\app.py
system_core\ui_nicegui\window.py system_core\services\rclone_service.py
runtime\python.exe system_core\ui_nicegui\app.py --smoke
runtime\python.exe system_core\main.py pairs
install\Check-CmdEncoding.cmd
install\Build-Docs-PDF.cmd --dry-run
Tools\rclone\rclone.exe version
Tools\rclone\rclone.exe listremotes
```

A separate security check should fail closed: `runtime\python.exe`
`system_core\ui_nicegui\app.py --host 0.0.0.0 --smoke`.

Canonical Workbench labels

Workbench uses the same Audion Image Tools public vocabulary in every project. Its buttons always keep the same order and labels: **Source**, **Add file...**, **Target**, **Reset**, **Delete**, **List**.

Reset returns to project **input/output** and does not delete files; **Delete** clears the current **Source** and **Target** only after confirmation. The exact Russian labels are **Источник**, **Добавить файл...**, **Назначение**, **Сбросить**, **Удалить**, **Список**. The Workbench variants **Destination**, **Clear**, **Цель**, and **Очистить** are not used.

Detailed Operational Decisions

Backup Mirror Dry Run

Use the dry-run form before any mirror that can remove destination-only data. It should inventory both sides, resolve filters, calculate planned copies and deletions, and write a report without changing files. Review the destination path character by character, especially when a saved profile, mapped drive, removable disk, or UNC share is involved. A dry run is valid evidence only when it uses the same Source, Target, filters, and mode as the intended real run.

The report must distinguish new, changed, unchanged, excluded, destination-only, inaccessible, and conflicting items. Large destination-only counts are a stop signal until the operator confirms that the correct destination and filter set were selected. Do not promote a dry run to execution by manually retyping paths; use the reviewed state or saved profile so scope does not drift.

Backup Mirror And Quarantine

The normal mirror sequence copies required data, verifies it according to the selected policy, and only then handles destination-only items. Quarantine mode moves those items into a dated recovery area instead of deleting them immediately. Quarantine consumes extra space but is preferred when the destination contains hand-maintained material, previous versions, or data whose ownership is uncertain.

Keep the quarantine manifest with its files. Before removing a quarantine set, confirm that the accepted mirror opens correctly, expected counts and hashes match, and no excluded folder was unintentionally treated as obsolete. A mirror is not a backup history by itself; if previous states matter, combine it with versioned archives or a filesystem that retains snapshots.

One-Way Synchronization

A one-way run copies new or changed items from Source to Target without automatically making the target an exact mirror. Use it for publishing, replenishing a secondary store, or transferring additions when destination-only data must remain. Run its dry-run variant first when overwrite rules, timestamps, or filters have changed.

Define whether a difference is detected by size and time, checksum, or tool-specific comparison. Timestamps can drift across filesystems, network shares, cameras, archives, and cloud providers. When data integrity matters more than speed, use checksum verification for the accepted set and record the algorithm in the report.

Compare Only And Manifest Diff

Compare-only mode creates evidence without copying or deleting. It is useful for acceptance, migration checks, and diagnosing why two trees differ. Manifest/diff workflows create durable inventories that can be compared later or on another machine. Include normalized relative paths, sizes, timestamps, and optional hashes; record exclusions and case-sensitivity assumptions.

A manifest is not proof that every file was readable after it was written. For long-term storage, pair inventory with sample or full readback, archive tests, storage health checks, and an independent copy. Preserve old and new manifests together when they are used to prove a migration.

Two-Way Synchronization

Two-way sync is appropriate only when both roots are intentional writable peers and conflict behavior is understood. Run dry-run first, inspect conflicts, clock differences, renames, and deletions, then

resolve ambiguous items before execution. Never point two-way sync at an authoritative source and a disposable output folder.

If the tool cannot prove a rename, it may appear as one deletion and one addition. If both sides changed the same relative file, automatic newest-wins behavior can destroy the wanted version. Preserve conflict copies or quarantine, keep both source roots backed up, and document the decision used for each important conflict.

Archive Operations

Archive creation should state the source scope, format, compression level, split-volume policy, encryption setting, and destination. Test the final archive with the same tool family expected for recovery. For encrypted archives, verify recovery from a separate process and store credentials through an approved channel; an unreadable encrypted backup is equivalent to no backup.

Extraction is a write operation. Inspect archive paths, available space, overwrite policy, and possible path traversal before extracting untrusted content. Extract to a new folder for review rather than over an accepted tree. Multi-volume archives require every part and stable filenames.

Network Operations

For UNC paths and mapped drives, confirm identity and availability before launch. A mapped drive can resolve differently in an elevated process, scheduled task, service account, or another user session. Prefer recording the UNC source when reproducibility matters. Network interruption should produce an explicit failed or incomplete state, not a successful report based only on the child process having exited.

Measure throughput over time and watch for retries, reconnects, and server-side throttling. Do not assume quiet output means a hung job; check process state, destination growth, log timestamps, and provider status. Resume only when the selected mode and tool guarantee safe idempotent behavior.

Rclone And Cloud Providers

Rclone profiles separate provider credentials and remote configuration from the chosen file operation. Confirm the remote name, bucket or root, Source/Target direction, encryption layer, checksum support, and server-side capabilities before a large run. Some providers do not expose the same hashes as local files, and some modify timestamps or metadata.

Start with list or compare operations, then a small controlled transfer. Review provider errors, rate limits, API quotas, multipart behavior, and eventual consistency. When a cloud operation is used as

backup, test download and open a representative set; successful upload counters alone do not prove recoverability.

Profiles, Filters, And Reports

A saved profile should capture operation type, roots, filters, comparison policy, overwrite/delete behavior, verification, quarantine, and tool-specific options. Treat profile changes as configuration changes: review them, give them clear names, and keep the selected profile version in the report.

Extension filters should be explicit about inclusion and exclusion, case handling, compound extensions, files without extensions, hidden/system files, and directory rules. Test a filter against a representative tree before relying on it for deletion or archival completeness.

The final report should identify every root and mode, started/finished time, tool versions, selected profile and filters, item/byte totals, errors, skipped/conflicting items, verification method, quarantine path, and exit state. Preserve the report with accepted output so later cleanup or migration is based on evidence rather than memory.

Choosing The Correct Transfer Mode

Use compare or manifest/diff when the first requirement is evidence and no files should move. Use one-way sync when one side is authoritative. Use backup mirror only when deletion semantics and destination ownership are understood. Use two-way sync only with a tested conflict policy. Archive mode packages files but does not replace synchronization or backup verification.

For remote and cloud routes, separate transport from synchronization semantics. SMB, Robocopy, 7-Zip transfer, and rclone can move the same data while preserving very different metadata and deletion behavior. Select the route first, then review overwrite, delete, retry, checksum, bandwidth, and resume parameters.

Filters And Presets

Treat the filter field as a controlled file browser. Confirm include and exclude patterns against a dry listing before a destructive sync. Saved pairs and presets store convenience state; they do not prove that the currently mounted drives or remote endpoints are the same devices used when the preset was created.

After loading a preset, verify both resolved paths, direction, mode, filter, overwrite policy, and delete policy. Network drive letters and removable-disk letters can change between Windows sessions.

Verification After A Run

Review the final process exit, the command log, copied/deleted/error counts, and any generated manifest or checksum. For a migration or backup, open representative files from the destination and compare the total file count and size with the planned scope.

If a long operation appears silent, inspect the child process, output timestamps, network or disk activity, and the current log. Do not kill a transfer solely because the parent terminal has not printed a new line.

High-Risk Actions

Mirror deletion, partition/disk operations, cloud purge, and destination cleanup require a separate confirmation and a verified target path. Never construct a delete target from unvalidated text. Avoid running through reparese points or unexpected mount paths. Preserve the latest accepted backup until the replacement has passed verification.

Recovery

On failure, keep partial output and logs until the cause is known. Resume only when the selected tool guarantees compatible continuation; otherwise restart into a clean destination or use a new output folder. Do not automatically reverse a partially completed two-way sync.

Detailed GUI Reference

This GUI is a file-manager workbench over the existing CLI. It does not duplicate the file engine or keep a separate business layer: sync operations become launches of `system_core\main.py`, while GUI-only workflows such as archiving use the same portable runtime and show output in the right terminal panel.

The documentation stack starts in this guide. The complete GUI/CLI command and parameter reference is `COMMAND_REFERENCE_RU.md`. Workbench, terminal, cache, and tooltip behavior are covered in `WORKBENCH_AND_GUI_RU.md`.

The main model is simple: choose a **Source** and a **Target**, then run the needed mode over that route. The route can point to project `input/output`, an external disk, a UNC share, a mounted folder, a

cloud-client directory, or any other folder visible to Windows/Python. Large folders do not have to be copied into the project first.

Launch

```
launcher_gui.cmd
```

The desktop wrapper chooses the next free local port if the default `8080` is already busy.

The GUI with the right-side command terminal is allowed only on loopback hosts: `127.0.0.1`, `localhost`, or `::1`. Starting it on `0.0.0.0`, a LAN/VPN address, or another non-loopback host is refused because the web panel can execute commands. A deliberate remote mode requires `AUDION_ALLOW_REMOTE_GUI=1`; treat that as an RCE surface, not a normal operating mode.

The desktop wrapper no longer requests administrator rights by default. Use `--elevate` or `AUDION_ELEVATE=1` only for procedures that explicitly need UAC.

If pywebview is not available, open the server part in a browser:

```
runtime\python.exe system_core\ui_nicegui>window.py --browser
```

Window Layout

Left column:

- current **Source** and **Target** with Pin, Unpin, and open-folder icons;
- folder pickers for arbitrary source and target paths;
- source file listing;
- operation tree;
- parameters for the selected mode, including filters and archive options.

Right column:

- current status;
- progress;
- live CLI terminal output;
- `Logs`, `Reports`, `Latest`, `Settings` buttons;

- opening the latest report.

Header:

- theme selector from `config\ui_colors.yaml`;
- RU/EN switch;
- cancel button, shown only while an operation is running.

All GUI tooltips use one timing rule: show after `1500 ms`, hide after `100 ms`, transition duration `100 ms`. Native browser `title` tooltips are disabled to avoid a double tooltip layer; controls keep `aria-label` for accessibility. The shared visible tooltip is rounded, uses `RGB(23, 33, 43)`, a thin border, and compact readable text.

A thin splitter between the left side and the terminal changes the right-panel width. The selected width is stored locally in the GUI browser profile.

Terminal Output

The right panel is a read-only live terminal, not an interactive shell. Text can be selected and copied.

ANSI colors are rendered through safe HTML: text is escaped, only allowed SGR codes are supported, and control noise is discarded. Python commands use UTF-8 for Cyrillic output, while Windows utilities are decoded through OEM/cp866/cp1251 fallbacks.

The visible terminal wraps long lines and keeps up to 2000 lines for the current operation. Full logs and large file lists are saved as ordinary UTF-8 text without ANSI escape sequences.

The command bar under the terminal keeps up to 200 commands in

`config\terminal_commands.json`, supports pins, shell selection, and working-folder selection. The command field starts empty; previous commands are selected from history. `History` clears unpinned history, while `Cache` resets history, pins, and last command while keeping shell and working folder. Commands that look sensitive (`token`, `secret`, `password`, `Authorization`, `Bearer`, `-p...`, and similar markers) are not saved or pinned.

Working Route

The top panel shows two current folders:

- **Source** - where files are read for listing, sync, audit, and archiving.
- **Target** - where operation results, archives, and some reports are written.

Each path tile has three compact icons:

- **Pin** stores the current path in the local cache;
- **Unpin** removes the pin;
- the folder button opens the path in Explorer.

The path itself opens a wide cache dropdown. Pinned entries are shown first and marked with **PIN |**.

The lower-row **Source** and **Target** buttons open the current folders. **Reset** returns the route to project **input/output** without deleting files. After a separate confirmation, **Delete** clears the current **Source** and **Target**, including external routes.

On startup the Workbench also starts from project **input/output**. Selected routes are remembered only in the local **config\path_history.json** cache, so portable settings do not leak machine-specific paths.

List builds a preview of files found in the current source. If a saved profile or manual extension filter is active, the list uses the same filter, preset, and masks that the operation will use. Without an active filter, all source files are shown. The full list is saved under **report**, and the terminal shows a quick preview.

Extension Filters

Operations with extensions use one shared mask set with **INCLUDE** and **EXCLUDE** tabs. **INCLUDE** defines include/mask globs. **EXCLUDE** stores a separate exclusion configuration and removes selected masks after profile/include filtering. Exclusions are passed to the CLI as **--exclude-globs**.

DATA TRANSFER, **FOLDER TO FOLDER** and saved profiles share the same pinned filter surface: Pin/Unpin, delete cached set, clear selection, cached-set dropdown, and a four-line summary for include/exclude groups and point masks.

Extension grids follow one strict layout rule: three cards in a row have equal width, two cards split the full row in half, and a single or very long group takes the full row. Inside a card, 1-3 checkboxes stay vertical; 4 or more options switch to two internal columns. This keeps sections such as **Video** and **Audio** aligned instead of looking like a random set of card widths.

Masks is a manual mode for one-time or repeatable extension sets and also a glob generator for external tools. A comma-separated input like **jpg, .PNG, *.webp** is normalized to ***.jpg, *.png, *.webp**. Mask sets are saved in **config\mask_cache.json**.

Main Modes

ROOT contains: `DATA TRANSFER`, `FOLDER TO FOLDER`, `Masks`, `Saved profiles`, `Archiving`, `Manifest / Diff`, `PREPARATION`, and `Maintenance`.

`PREPARATION` contains dry runs, compare-only reports, and quarantine variants for manual routes and saved profiles. Working profiles open directly without an intermediate child menu.

BACKUP-MIRROR Dry Run

Shows what the selected backup policy would do:

- files copied from source to target;
- files updated;
- target-only files moved to quarantine or deleted under `MIRROR_HARD`.

No files are changed.

BACKUP-MIRROR

Mirror backup mode. By default this is a hard mirror: copy/update, verify, then delete target-only files only if earlier phases succeed.

`MIRROR_SAFE` stays available when quarantine is preferred over direct delete.

BACKUP-MIRROR Quarantine

Copies and updates like `BACKUP-MIRROR`, but moves target-only files to `_audion_quarantine\YYYYMMDD_HHMMSS_microseconds`, preserving relative paths.

One-Way Sync

Dry run shows which new and changed files would be copied from source to target. Real sync copies those files and leaves target-only files untouched.

Compare Only

Builds a difference report without changing the disk. Use it before important or large runs when you need to inspect folder differences first.

Two-Way Sync

Copies missing or newer files both ways and does not delete automatically.

Manifest / Diff

Manifest / Diff works over the current Workbench **Source -> Target** route. Leave the extension filter empty to scan all files, or select groups/extensions to narrow the diff. **ONLY DIFF** writes relative path lists to **report**; **DIFF + ONE-WAY SYNC** and **DIFF + BACKUP-MIRROR** rebuild the diff and immediately apply it.

The source/target checksum buttons create or verify standalone **__CHECKSUMS__.b3** files. They are useful for integrity audit, but are not required for Diff + One-Way.

Archiving

Archiving reads items from the current source and creates results in the current target.

Supported formats:

- ZIP;
- 7Z;
- SFX **.exe**;
- TAR;
- TAR.GZ;
- TAR.ZSTD through portable PeaZip **zstd.exe**;
- optional TAR.LZ4 through a standalone **lz4.exe**, installed by the **INSTALL LZ4** button in the archive panel or by step **[06] LZ4** in **builder_main.cmd**.

The archive layout row uses inline radio chips: **TARGET** writes archives directly into the current target, while **TARGET/<name>** creates one subfolder per source item. **Verify archives after compression** tests each result before optional source deletion; **Delete source files and folders** runs only after successful creation and, when enabled, successful verification.

Encryption is explicit. **Do not encrypt** is the default. If a backend cannot encrypt or a format is unavailable in the portable bundle, the format checkbox is disabled. Password fields are masked, and archive **-p...** arguments are shown in the GUI log as **-p[REDACTED]**. The password is still passed to the CLI archiver as a process argument, so avoid typing it into the terminal command bar and do not use shared machines for private archives.

SFX mode can use Windows environment variables for the auto-extraction path. The UI shows an absolute preview for the current machine, while the SFX stores a portable path such as

`%LOCALAPPDATA%\Audion\<name>`. After SFX compression, the `.exe` can also be wrapped into ZIP/7Z/ZSTD/TAR/GZ with level 0 for easier transport.

Data Transfer

There used to be two sections: `Network operations` for LAN, SMB and UNC paths, and `Rclone operations` for remotes such as `drive:path`. Choosing a button meant knowing in advance which tool could reach where. They are now one section, `DATA TRANSFER`, where a folder, a share, a server and a cloud are equals.

You choose the action, not the tool. There are five operations: `One-way`, `Mirror`, `Move`, `Two-way`, `Verify`. The engine follows from each pair and is named on screen in its own line — Robocopy for folders and shares, RClone for clouds, two-way sync and hash comparison.

`Mirror` and `One-way` are different operations, not one with a deletion checkbox. A mirror's destination is a copy and owns nothing: what changes on the reference machine reaches every replica, deletions included. A one-way destination has a life of its own, and making it an exact copy of the source would wipe what already lives there.

The destination is a list, not a single field: one reference machine to four replicas is one run. The whole fan-out is built before anything starts, so an impossible pair shows up before the first byte moves.

`Pack first` archives the source and sends archives instead of loose files. On many small files over a network this is the only thing that helps: measured at 24 MB/s for a single file against 0.46 MB/s for three hundred small ones, at around 20% CPU — the time goes not into transfer but into the network greeting every file separately.

`Dry run` is available for every operation and writes nothing. The delete ceiling (`--max-delete`) is available and never imposed.

Details — operations, engine selection, masks, Robocopy exit codes — are in [TRANSFER_RU.md](#).

Connections, Storage and Diagnostics

The same section also holds everything around a transfer: connections (OAuth providers, S3-compatible storage by key, SFTP, config encryption), storage questions (size, duplicates, checksums, public links), and diagnostics.

The section is grouped into `Transfer`, `Storage`, `Connections` and `Diagnostics`. The `RClone config` block appears only on the `Settings` tab in `Config` mode, so `Version`, `List remotes`,

GUI, OAuth, and SFTP do not carry the installer/config-manager block with them. Its **Installer** **RClone** row has only **System** and **Portable**. **System** runs `install\Install-System-Rclone.cmd` and installs user-scope RClone into `%LOCALAPPDATA%\Programs\rclone`; **Portable** runs `install\Install-Portable-Rclone.cmd` and updates only `Tools\rclone`. Neither installer overwrites `rclone.conf`. **Config** and **GUI** open directly in a separate visible console without an intermediate shell string; normal commands stream into the right terminal.

The **Configuration** row has the config-scope selector **Portable**, **System**, **Custom path**, the custom config path field, and **Import**, **Export**, **Doctor**. **Custom path** is a path to a specific `rclone.conf`, not an install target. Import copies system `rclone.conf` to portable config with backup, export copies portable config to system config with backup, and doctor performs a read-only active config/remotes/absolute-path check. The default scope is **Portable**, so commands get `--config <ROOT>\config\rclone\rclone.conf` and `--cache-dir <ROOT>\tmp\rclone-cache`.

RClone modes are not shown as one long dropdown. The user first chooses a branch through the tab row: **Transfer**, **Route**, or **Settings**; only the commands for that branch remain visible below. The active command is highlighted with a subdued success color. A short pipeline summary remains below the buttons, and each mode tooltip explains the path, for example `Source -> remote:path -> rclone copy`, `remote -> remote -> rclone copy`, or `remote name + backend -> OAuth window -> remote:path`.

Bandwidth limit is a full-width row under the RClone flags. It offers **Unlimited**, **Speed preset**, and **Custom speed** with spinners; only one value source is active at a time, so preset/custom values do not stack.

Upload uses the current Workbench **Source** and builds `rclone copy <Source> <remote>: <path>`. Download uses the current Workbench **Target** and builds `rclone copy <remote>: <path> <Target>`. Check uses `rclone check <Source> <remote>:<path> --one-way` and writes a combined report into **logs**.

Universal routes use paired **SOURCE ENDPOINT** and **TARGET ENDPOINT** toggle panels. The primary scenario is saved `remote:path` on both sides, for example `drive:Project -> server:/home/user/backups` or `server:/home/user/backups -> drive:Restore`. Workbench Source/Target remain explicit endpoint type choices when a local side is needed.

Authorization lives in the **REMOTE** toggle panel. SFTP is built as `user@host:port` with **ssh-agent**, **password**, or **key_file**; OAuth/provider backends open a clear RClone wizard window. Full **Config** remains available for rare backend cases.

In `OAuth providers`, the default provider is Google Drive and the default `Remote name` is `drive`. `Remote name` follows the selected provider while it is empty or still a default name such as `drive`, `yandex`, or `dropbox`. If the user typed a custom name, the GUI preserves it.

Tooltips on `Remote name`, `Provider`, SFTP fields, and `RUN` must explain the exact action: which window opens, what to type into prompts, where Enter/default is expected, and which `remote:path` is produced after success. For Google Drive, the normal path is remote name `drive`, blank `Client ID`/`Client Secret`, advanced config default/No, auto config/browser Yes, then allow access in the browser.

SFTP `user` and `host/IP` fields have a history dropdown from successful recent operations. The cache lives in `config\rclone_endpoint_history.json`, contains no passwords, and is cleared by the clear button in the RClone auth panel. Documentation examples use the reserved example IP `203.0.113.10`.

The bottom of the form has a compact constructor: executable, route, config/cache, source/target, log, and the full command preview in small monospace text. Command templates can be pinned, unpinned, and deleted from cache; OAuth tokens and SFTP passwords are not stored in project JSON/YAML. Portable `rclone.conf` is kept as an ignored file under `config\rclone`.

Use `rclone_log_level=DEBUG` only for local diagnostics: the raw rclone log may contain sensitive remote/request details. Keep `INFO` for normal operations.

On slow links, the top progress bar/status is updated from standard rclone stats: `Transferred`, `Checks`, `Retries`, `Errors`, speed, and ETA. The right terminal still shows the full rclone stream without shortening it.

Saved Profiles

Profiles live in `config\sync_pairs.json`; filters live in `config\sync_presets.json`.

Saved profiles are direct actions in the GUI: selecting `dev_backup`, `docs_backup`, or another profile opens the launch form. Their `dry_run`, `compare`, and `quarantine` variants are in `PREPARATION`.

Current convention:

- `*_backup` examples use hard Backup mirror by default;
- `MIRROR_SAFE` is available through the quarantine button or explicit `operation_policy`.

The old `dry_run_default` key is ignored with a deprecation warning. Use `operation_policy: copy_update`, `mirror_safe`, or `mirror_hard`.

Reports

Main artifacts:

- `output\` - compare/sync/backup reports;
- selected **Target** - archives and GUI archiving results;
- `report\` - GUI/run artifacts;
- `logs\` - GUI and launcher logs.

`Latest report` in the right panel tries to open the newest human-readable report from `output\` or `report\`.

Appearance

GUI settings are split between:

- `config\ui_colors.yaml` - palettes, CSS tokens, fonts, and themes;
- `config\gui_settings.yaml` - start language, selected theme, emoji flag, and GUI-only settings without machine-specific source/target paths.

Internal standard:

- default theme is `code_dark`;
- theme switching from the header reloads the interface;
- compact buttons without filled backgrounds;
- muted borders;
- mode descriptions next to buttons;
- terminal on the right as a continuation of the older CMD/FZF workflow.

Cleanup And Folder Structure

`install\init_folders.cmd` creates all managed folders expected by CLI and GUI: `input`, `output`, `logs`, `report`, `workspace`, `data`, `runtime`, `wheelhouse`, `release`, `._runtime`, `system_core`, `install`, and `licenses`.

`input` and `output` remain convenient defaults and staging zones, but they are no longer the required working model. For large folders, choose an external source and target from the top panel.

`cleanup_project.cmd` clears only managed working/runtime folders and caches. Sources, documentation, configs, and licenses remain in place.

Safe Ladder

For important folders:

1. Run `Compare only`.
2. Run a dry run of the selected mode.
3. Review the report, quarantine candidates, and hard-delete candidates.
4. Run the real operation.

For ordinary backup work, the usual path is:

1. `One-way` / `COPY_UPDATE`.
2. `BACKUP-MIRROR` when a true mirror with delete after successful copy/verify phases is needed.

Canonical Workbench labels

Workbench uses the same Audion Image Tools public vocabulary in every project. Its buttons always keep the same order and labels: **Source**, **Add file...**, **Target**, **Reset**, **Delete**, **List**.

`Reset` returns to project `input/output` and does not delete files; `Delete` clears the current `Source` and `Target` only after confirmation. The exact Russian labels are **Источник**, **Добавить файл...**, **Назначение**, **Сбросить**, **Удалить**, **Список**. The Workbench variants `Destination`, `Clear`, `Цель`, and `Очистить` are not used.